



UFRN

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

**BENTO DE NEGREIROS BARJUD
ÉLISSON RIAN FÉLIX SIMAS
JORGE LUCAS LOPES DE CARVALHO
MURILO KONZEN**

RELATÓRIO DO PROJETO FINAL DE SISTEMAS DIGITAIS: PROCESSADOR 16 BITS

**NATAL- RN
2026**

1. DESCRIÇÃO DO PROJETO APROVADA PELO PROFESSOR:

“Eu, Jorge Lucas Lopes de Carvalho, e meu grupo (Bento de Negreiros Barjud, Élisson Rian Félix Simas e Murilo Konzen), pensamos em elaborar para o projeto RTL que consiste em um processador de 16 bits que realiza as operações de soma, subtração, multiplicação por 2, divisão por 2, and bit a bit, or bit a bit, load, store, add imediato, load imediato, comparador e clear.”

(e-mail, enviado pelo grupo, 2026)

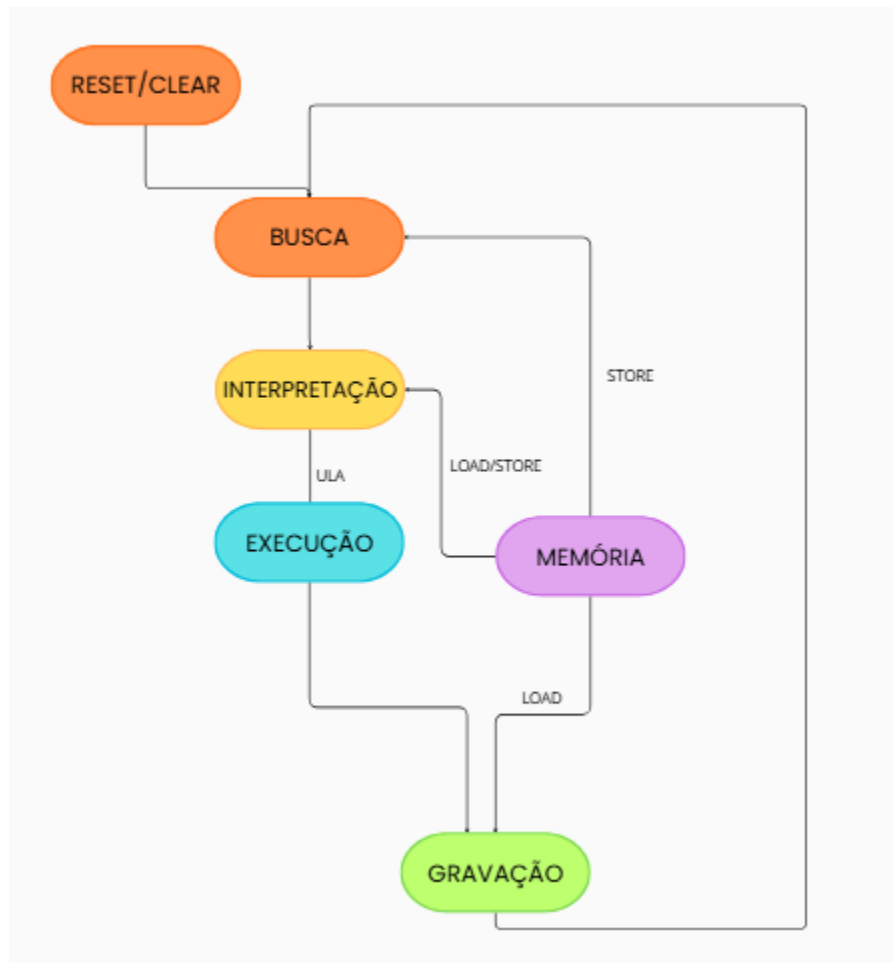
2. MÁQUINA DE ESTADO DE ALTO NIVEL

Tabela de sinais de cada estado

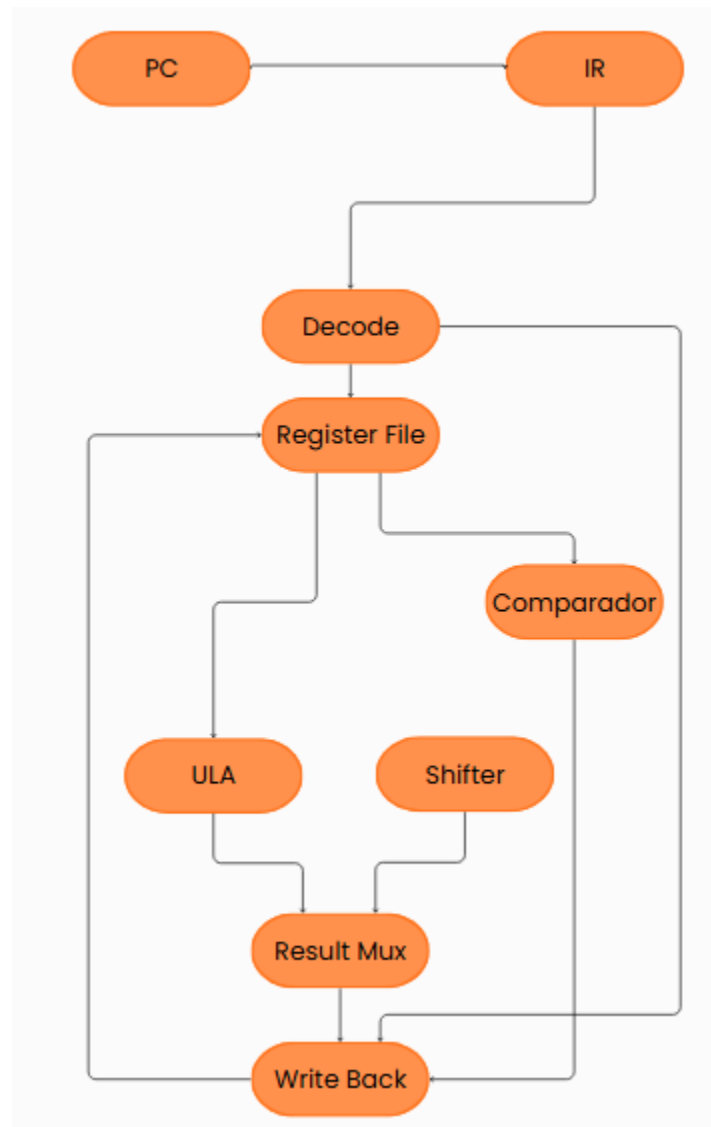
Estado	rf_wr	pc_inc	ir_ld	mem_rd	mem_wr
Busca	0	1	1	0	0
Interpretação	0	0	0	0	0
Execução	0	0	0	0	0
Memória	0	0	0	Load	Store
Gravação	1	0	0	0	0

O processador segue um ciclo de execução com 5 estados principais mais o estado de reset. A primeira etapa desse ciclo é responsável por receber a instrução, que irá ditar os processos seguintes, e somar um ao contador de programa (etapa de BUSCA), a instrução passa então a um estado de interpretação, que define os caminhos/próximas etapas (etapa de interpretação) caso a operação seja lógica/aritmética o quem recebe as informações é a unidade lógica aritmética (ULA) que trata os dados de acordo com o objetivo do comando, caso seja uma operação de load ou store, os dados serão gravados ou carregados (Memória). O Resultado é armazenado no registrador de destino ou na memória, caso seja um store. E, por fim, o contador do programa irá retornar ao estado de busca para encontrar a próxima instrução. Esse ciclo se repete continuamente enquanto o processador estiver em funcionamento.

Grafo de estados do processador



3. CAMINHO DE DADOS



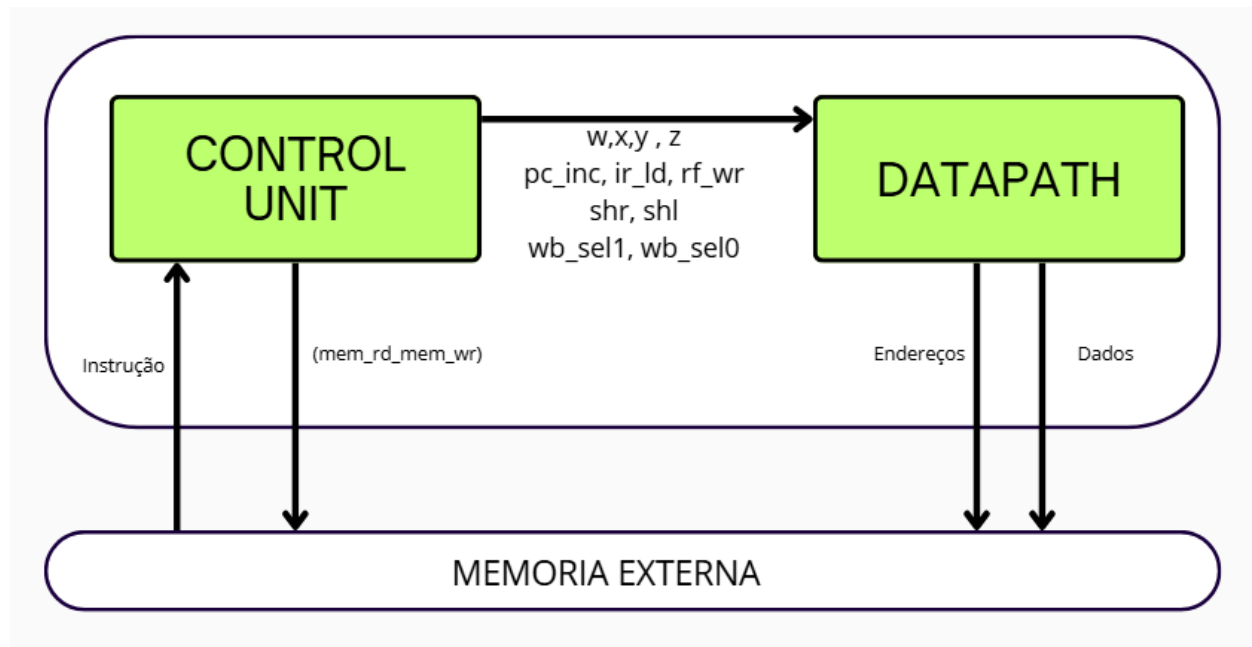
Grafo da arquitetura datapath

O projeto do caminho de dados (datapath) deste processador opera com barramentos de 16 bits e foi desenvolvido de forma estritamente estrutural. Seus principais componentes incluem o Program Counter (PC) de 16 bits, que possui um somador dedicado para a realização do incremento de $PC++$, e o Instruction Register (IR), que tem a função de armazenar a instrução atual. A arquitetura também comporta um banco de 16 registradores de arquivos endereçáveis por 4 bits, configurados para manipular dados de 16 bits. É a partir das saídas deste banco que os barramentos de memória são alimentados diretamente: o endereço de memória de dados é retirado do registrador RA, enquanto o dado a ser gravado em operações de STORE é retirado do registrador RB.

Para executar as mais diversas operações sem recorrer a lógicas comportamentais, os módulos de processamento foram desenhados separadamente. A Unidade Lógica e Aritmética (ULA) foi dividida estruturalmente, instanciando um extensor lógico (ELA) que prepara os operandos puramente através de equações de soma de produtos, enviando-os na sequência para um somador ripple-carry de 16 bits. Em paralelo à ULA, o projeto inclui módulos auxiliares independentes: um Shifter para os deslocamentos que realizam multiplicação e divisão por 2, e um Comparador para as avaliações de magnitude. O comparador, por sua vez, realiza obrigatoriamente a comparação entre as portas A e B (RA e RB) do banco de registradores.

Por fim, o roteamento dos dados de volta ao banco é administrado por uma cascata estrutural de multiplexadores. Em vez de implementar uma única grande estrutura de seleção, o hardware faz uma escolha prévia entre a ULA e o Shifter para gerar o barramento intermediário. Somente após essa etapa o resultado entra na porta 00 do Multiplexador de Write-Back final. Esse MUX 4x1 de Write-Back, inteiramente combinacional e implementado bit a bit com equações booleanas massivas, atua como o responsável por decidir o que será definitivamente escrito no registrador de destino. Ele concretiza a seleção roteando entre o resultado da ULA/Shifter (00), os dados vindos da Memória (01), um valor Imediato (10) ou o resultado de uma Comparação (11).

4. PROJETO COMPLETO EM RTL



Grafo da arquitetura RTL

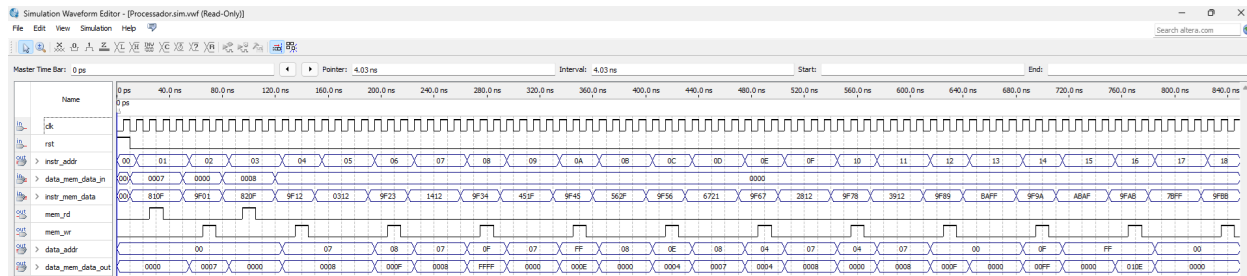
O desenvolvimento do projeto RTL (Register-Transfer Level) baseia-se em uma arquitetura de processador multi-ciclo de 16 bits, otimizada para operações do tipo Load/Store. Um pilar central deste design é a restrição arquitetural adotada: descrições comportamentais em VHDL foram restritas à modelagem de elementos de estado, como Flip-Flops e registradores, enquanto todo o controle e roteamento de dados foram implementados através de lógica combinacional estrutural ou fluxo de dados. Essa estratégia garante a execução precisa de operações aritméticas, lógicas, acessos à memória e manipulação de valores imediatos.

A hierarquia do sistema é centralizada no módulo "Processador", que atua como o topo da hierarquia, integrando a Unidade de Controle (FSM) e o Caminho de Dados (Datapath). A Unidade de Controle, ao receber o opcode (bits 15 a 12) fornecido pelo Datapath, gerencia a execução dos ciclos ao disparar os sinais de controle essenciais, tais como o incremento do PC, a habilitação de escrita em registradores e a configuração dos seletores de multiplexadores. Essa separação clara entre blocos sequenciais e componentes combinacionais confere ao design uma estrutura robusta e uma máquina de estados altamente previsível.

A interface com o sistema de memória é realizada com o processador exportando barramentos de endereços e dados, além de sinais de controle dedicados para leitura e escrita. Essa configuração permite uma integração direta com módulos externos de memória de instrução e de dados. A validação funcional, confirmada pela análise das formas de onda, assegura a correta decodificação das instruções e a sincronização dos acessos aos barramentos conforme os requisitos de subida do clock.

5. WAVEFORM E DESCRIÇÃO DAS OPERAÇÕES

Waveform com a simulação de todas as instruções possíveis



Sequência de instruções em hexadecimal utilizadas:

- | | | |
|---------------------------------|-----------------------------------|-----------------------------------|
| (01) LOAD [810F]
R1 = 7 | (09) SHL [451F]
R5 = R1 << | (11) OR [3912]
R9 = R1 R2 |
| (02) STORE [9F01]
D0 = R1 | (0A) STORE [9F45]
D255 = R5 | (12) STORE [9F89]
D0 = R9 |
| (03) LOAD [820F]
R2 = 8 | (0B) SHR [562F]
R6 = R2 >> | (13) LOADI [BAFF]
R10 = FF |
| (04) STORE [9F12]
D7 = R2 | (0C) STORE [9F56]
D14 = R6 | (14) STORE [9F9A]
D15 = R10 |
| (05) ADD [0312]
R3 = R1 + R2 | (0D) CMP [6721]
R7 = R1 >=< R2 | (15) ADDI [ABAF]
R11 = R10 + F |
| (06) STORE [9F23]
D8 = R3 | (0E) STORE [9F67]
D4 = R7 | (16) STORE [9FAB]
D255 = R11 |
| (07) SUB [1412]
R4 = R1 - R2 | (0F) AND [2812]
R8 = R1 && R2 | (17) CLR [7BFF]
R11 = 0 |
| (08) STORE [9F34]
D15 = R4 | (10) STORE [9F78]
D4 = R8 | (18) STORE [9FBB]
D0 = 0 |

As primeiras 4 instruções passam, os valores 7 e 8 para os registradores R[1] e R[2], e armazenam elas em endereços da memória (nesse caso a memória não foi implementada, assumindo valores de entrada arbitrários, “forçados” na simulação e valores que não são de fato armazenados na saída).

Os comandos 05, 07, 09, 0B, 0D, 0F e 11 todas as operações lógicas e aritméticas implementadas e seguem, essencialmente o mesmo padrão de comando, OP_CODE(define qual operação está sendo realizada), Registrador onde o resultado será gravado (RD), Registrador onde primeiro valor está gravado (RA), registrador onde o segundo valor está armazenado (RB).

Em binário:

OP₃ OP₂ OP₁ OP₀ RD₃ RD₂ RD₁ RD₀ RA₃ RA₂ RA₁ RA₀ RB₃ RB₂ RB₁ RB₀

Como exceção a esse formado, temos os procedimentos de shift, que ignoram o endereço do segundo operador (RB),

Na simulação apresentada, as operações utilizam os valores armazenados em R1 e R2 (07 e 08), e todas apresentam as saídas esperadas.

Além dessas dos comandos matemáticos, existem instruções que interagem diretamente com a memória e registradores representados pelas linhas 01, 02, 03, 04, 06, 08,0A, 0C, 0E, 10, 12, 13, 14, 15, 16, 17 e 18. As operações de LOAD e STORE consistem, respectivamente, na atribuição a um registrador de um valor contido em um endereço específico de memória, no armazenamento do valor de um registrador em um endereço de memória. Quando aos comandos restantes, temos o LOADI (Load Imediato) que atribui um valor diretamente a um registrador, sem consultar na memória, utiliza para isso os últimos 8 bits; ADDI (adição imediata) , guarda em um registrador o resultado da soma entre o valor de um registrador e uma palavra de 4 bits contida na própria instrução, por fim o CLEAR que atribui o valor 0 a um registrador.

Para a aplicação dessas instruções, utilizamos os resultados das operações aritméticas como os endereços de memória, é observável que os índices de D, que aparecem são convenientemente iguais ao resultado da operação anterior.

Tabela com a formatação das instruções

Operação	Equação	Instrução			
		OPCODE	RD	RA	RB
ADD	$RD = RA + RB$	0	D	A	B
SUB	$RD = RA - RB$	1	D	A	B
AND	$RD = RA \&\& RB$	2	D	A	B
OR	$RD = RA \parallel RB$	3	D	A	B
SHL	$RD = RA \ll$	4	D	A	DON'T CARE
SHR	$RD = RA \gg$	5	D	A	DON'T CARE
CMP	$RD = RA \geq \leq RB$	6	D	A	B
CLR	$RD = 0$	7	D	DON'T CARE	DON'T CARE
LOAD	$RD = RAM$	8	D	*RAM	DON'T CARE
STORE	$RAM = RB$	9	DON'T CARE	*RAM	B
ADDI	$RD = RA + IMM4$	A	D	A	IMM4
LOADI	$RD = IMM8$	B	D	IMM8	

*RAM: Registrador que armazena o endereço de memória que deverá ser lido ou gravado

IMM4: Valor a ser somado 4 bits

IMM8: Valor a ser armazenado 8 bits

6. DIFICULDADES E SOLUÇÕES

Um dos desafios centrais do projeto foi o gerenciamento de palavras de 16 bits na programação, dado o seu volume. Para mitigar esse problema, adotou-se a representação em hexadecimal, o que demandou conversões constantes entre as bases decimal, binária e hexadecimal para garantir a codificação precisa de cada instrução.

Além disso, a complexidade inerente à arquitetura exigiu um planejamento minucioso para realizar a integração harmoniosa entre a Unidade Lógica e Aritmética (ULA), o banco de registradores e o sistema de memória.

Houve também um obstáculo físico relacionado à escassez de pinos na FPGA utilizada. O plano inicial de mapear diversos sinais para simplificar os testes e a visualização em hardware foi limitado pela capacidade da placa. Como resposta, otimizou-se a alocação de I/Os, concentrando os recursos nos sinais fundamentais para comprovar o funcionamento do processador.

Tais dificuldades foram superadas por meio de uma rotina rigorosa de testes, refinamentos no design e ajustes na arquitetura. Esse processo permitiu que o processador fosse devidamente implementado e validado, apresentando resultados satisfatórios tanto no ambiente de simulação do Quartus quanto na prática com a FPGA.

7. LINKS:

[Vídeo da implementação na FPGA.](#)

[Repositório com o projeto em VHDL.](#)

8. CONCLUSÃO

Em última análise, o desenvolvimento do processador de 16 bits foi concluído com sucesso, demonstrando plena eficácia na execução das operações projetadas. Além dos resultados técnicos, a elaboração deste projeto foi fundamental para consolidar o entendimento teórico e prático dos integrantes sobre sistemas digitais complexos.

A transição bem-sucedida do modelo conceitual para a implementação em hardware proporcionou uma visão ampla de todo o ciclo de desenvolvimento. O desafio de gerenciar a complexidade do design RTL, superando limitações de tempo, foi crucial para o aprendizado sobre otimização de recursos e eficiência de projeto. A validação funcional, comprovada tanto na simulação quanto na prática, confirma a robustez da arquitetura desenvolvida. Por fim, este projeto não apenas cumpre os requisitos acadêmicos, mas também estabelece uma base sólida para futuras expansões, como a implementação de um conjunto de instruções mais extenso ou a otimização da interface de memória, consolidando conhecimentos essenciais para o desenvolvimento de sistemas digitais.